

## Rybina search (rybina)

Bei der Schueberfouer, dem grössten Jahrmarkt in Luxemburg, gibt es ein beliebtes Zahlenratespiel. Der Schausteller, Rybina, versteckt ein geheimes Passwort der Form  $AB$ , wobei  $A$  und  $B$  beides  $N$ -stellige Zeichenfolgen sind. Er erlaubt dir, zu versuchen, das Passwort zu erraten. Nach jedem Versuch sagt er dir, ob deine geratene Zahl zu hoch oder zu niedrig ist.

Nach jedem Versuch kann Rybina sich jedoch dazu entscheiden, die zwei Hälften des Passwortes zu tauschen. Das heisst, aus  $AB$  wird  $BA$  (bzw. aus  $BA$  wird  $AB$ ). Falls er das tut, wird er es dir sagen. Dein nächster Versuch wird dann gegen das vertauschte Passwort verglichen.

Deine Aufgabe ist es, das geheime Passwort in höchstens 64 Versuchen herauszufinden.

### Implementierung

Du musst eine einzelne `.cpp`-Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest du eine Vorlage `rybina.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktion implementieren:

|     |                                        |
|-----|----------------------------------------|
| C++ | <code>void rybina_search(int N)</code> |
|-----|----------------------------------------|

- Die Ganzzahl  $N$  repräsentiert die Anzahl an Ziffern, die jede Hälfte des Passwortes beinhalten.

Dein Programm kann die folgenden Funktionen aufrufen, die bereits im Grader definiert sind.

|     |                                             |
|-----|---------------------------------------------|
| C++ | <code>GuessResult guess(long long X)</code> |
|-----|---------------------------------------------|

- Die Ganzzahl  $X$  repräsentiert das Passwort, welches du geraten hast. Es sollte nicht-negativ sein und höchstens  $2N$  Ziffern beinhalten, wenn es in Dezimalform ausgeschrieben wird (i.e.  $0 \leq X < 10^{2N}$ ).
- Falls dein Versuch korrekt ist, wird dein Programm beendet und du bekommst die Bewertung `accepted` für den aktuellen Testfall.
- Ansonsten gibt die Funktion ein `GuessResult` zurück, welches ein Struct mit dem Ganzzahl-attribut `comparison` und dem boolsche Attribut `swapped` ist.
- Der Wert von `comparison` ist 1, falls dein Versuch zu tief war, und  $-1$  falls dein Versuch zu hoch war.
- Der Wert von `swapped` zeigt an, ob das Passwort nach deinem Versuch getauscht wurde.

Beachte, dass der Grader **adaptiv** ist und weder die richtige Antwort noch die Anzahl an Vertauschungen am Anfang bestimmt werden.

### Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die du verwenden kannst, um deine Lösung lokal zu testen. Der Beispielgrader liest die Eingangsdaten aus der Datei `stdin`, ruft die Funktion `rybina_search` auf und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus zwei Zeilen:

- Zeile 1: Die Ganzzahl  $N$ .
- Zeile 2: Die Ganzzahlen  $S$  und  $T$ . Die Erste,  $S$ , ist der Seed, welcher benutzt wird, um die zufälligen Vertauschungen zu bestimmen; Die Zweite,  $T$ , ist das Ziel: das originale passwort  $AB$ , geschrieben als eine einzelne Ganzzahl mit  $0 \leq T < 10^{2N}$ .

Nach jedem Versuch tauscht der Beispielgrader die beiden Hälften des Passworts zufällig mit dem Seed  $S$ . (Der Jury-Grader, welcher für die offizielle Bewertung verwendet wird, ist stattdessen adaptiv.)

Der Beispielgrader meldet das Resultat in `stderr`: Falls deine Lösung das Passwort korrekt erraten hat, gibt es die Anzahl an verwendeten Versuchen aus. Ansonsten gibt es eine kurze Begründung aus (ein ungültiger Versuch, oder zu viele Versuche).

## Einschränkungen

- $1 \leq N \leq 9$ .

Du kannst die Funktion `guess` höchstens 64 Mal aufrufen.

## Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Um die Punktzahl zu erhalten, die einer Teilaufgabe zugeordnet ist, musst du alle darin enthaltenen Testfälle korrekt lösen.

- **Teilaufgabe 0 [ 0 Punkte]**: Beispiel-Testfälle.
- **Teilaufgabe 1 [ 5 Punkte]**:  $N = 1$  und  $B = 0$ , i.e. die zweite Hälfte des originalen Passworts ist 0.
- **Teilaufgabe 2 [ 5 Punkte]**:  $N = 1$ .
- **Teilaufgabe 3 [10 Punkte]**:  $N \leq 4$  und Rybina tauscht das Passwort nie.
- **Teilaufgabe 4 [20 Punkte]**: Rybina tauscht das Passwort nie.
- **Teilaufgabe 5 [15 Punkte]**: Rybina tauscht das Passwort immer.
- **Teilaufgabe 6 [20 Punkte]**:  $N \leq 4$
- **Teilaufgabe 7 [25 Punkte]**: Keine weiteren Einschränkungen.

## Beispiele

| Grader                        | Lösung                   |
|-------------------------------|--------------------------|
| <code>rybina_search(2)</code> |                          |
|                               | <code>guess(1000)</code> |
| <code>return [1, 1]</code>    |                          |
|                               | <code>guess(1000)</code> |
| <code>return [-1, 0]</code>   |                          |
|                               | <code>guess(412)</code>  |
| <code>return [0, 0]</code>    |                          |

## Erklärung

Im ersten Beispiel,  $N = 2$  und die zwei Teile des geheimen Passwortes sind 12 und 04. Es wurde 1000 geraten. Zu diesem Zeitpunkt ist das Passwort 1204, also ist der Versuch zu tief. Nach diesem Versuch, vertauscht Rybina das Passwort. Das Passwort ist nun 412, und es wird wieder 1000 geraten. Dieses Mal ist der Versuch zu hoch. Rybina vertauscht danach das Passwort nicht. Schlussendlich wird 412 geraten, welches korrekt ist. Das Programm wird beendet.